Lecture 15: Chubby and ZooKeeper (Lock/Notification Services)

CS 739: Distributed Systems

Prof. Andrea C. Arpaci-Dusseau
University of Wisconsin — Madison

Today's Topic: Chubby and ZooKeeper

1. Chubby: Mike Burrows. The Chubby Lock Service for Loosely-Coupled Distributed Systems. OSDI 2006. SIGOPS Hall of Fame Award
2. Zookeeper: Patrick Hunt, Mahadev Konar, Flavio Paiva Junqueira, and Benjamin Reed. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In USENIX Annual Technical Conference (ATC), 2010.

Why Today's Papers?

- Chubby: Highly successful inside Google
- ZooKeeper: Influential outside (open-source)
- Interesting comparisons between the two
  - Consistency vs. Performance: Read only at leader vs. follower
- Interesting discussion of practical issues that arise
- How other distributed services use these services
- Techniques: Caching, leases, failure handling, consensus, API: ephemeral nodes

High-level Purpose of each System

Chubby

- Coarse-grain locks as highly-available service (w/ small amount of data)
- Synchronize client activities and agree on basic information

ZooKeeper

- Coordination service so developers can implement own primitives
- Chubby w/o blocking operations (no locks, open, close)

Use Cases across both Services?

Examples?

- Leader election
- Membership (leader discovers followers)
- Service discovery
- Meta-data (root of distributed data structure)
- Allocation of partitioned data
- Barriers to coordinate steps
- Configuration management
- Rendezvous service

Why not implement as Paxos library?

Paxos library linked with client

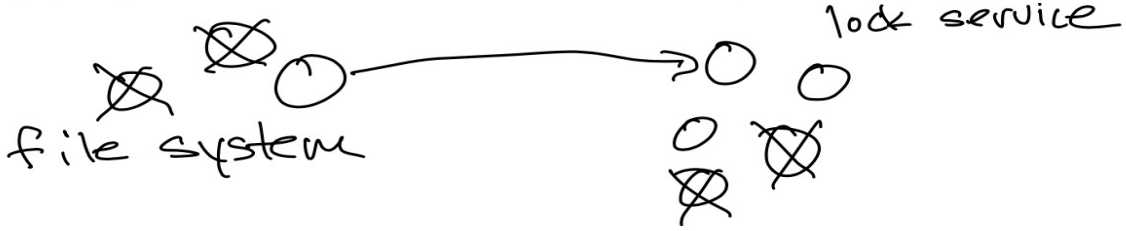
- \+ Completely general
- \- Difficult to write service as state machine

Separate Lock service

- \+ Easier to add incrementally when service needs availability and scale
- \+ Becoming master ←→ Acquiring exclusive lock
- \+ Need mechanism to advertise results

  Fetch small amounts of data
- \+ Locks are familiar
- \+ Separate availability of client service from that of lock service

  How many nodes need to be up to make progress?

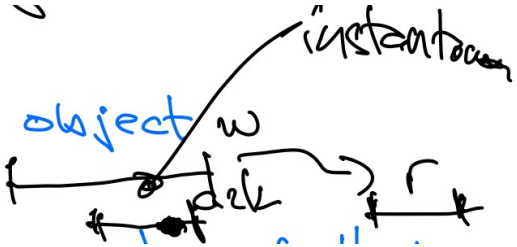


Lock Service: Assumptions

Design as Specialized Service

Always beneficial to understand workload and use cases

1) 1000s clients may read; **Implications?**
   - Cache data on clients
   - Need notification of changes (no polling)



2) Intuitive consistent caching
   - Linearizable
     - Guarantee provided for single object (not across different objects)
     - Write appear instantaneous
     - After completion of write, later reads return value of that write (or later write)
       All later reads return that value or value of later write

3) Security and access control

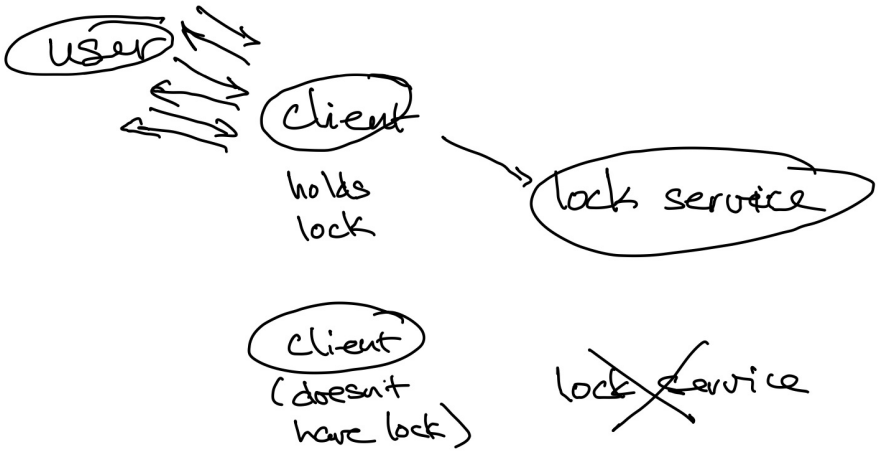
Prioritize performance or availability?

Lock Service: Coarse-Grain Locks

Purpose of locks is not to protect some small critical sections

- Instead: Get information or determine master for significant amount of work

Assume user interaction with client rate >> lock acquisition rate

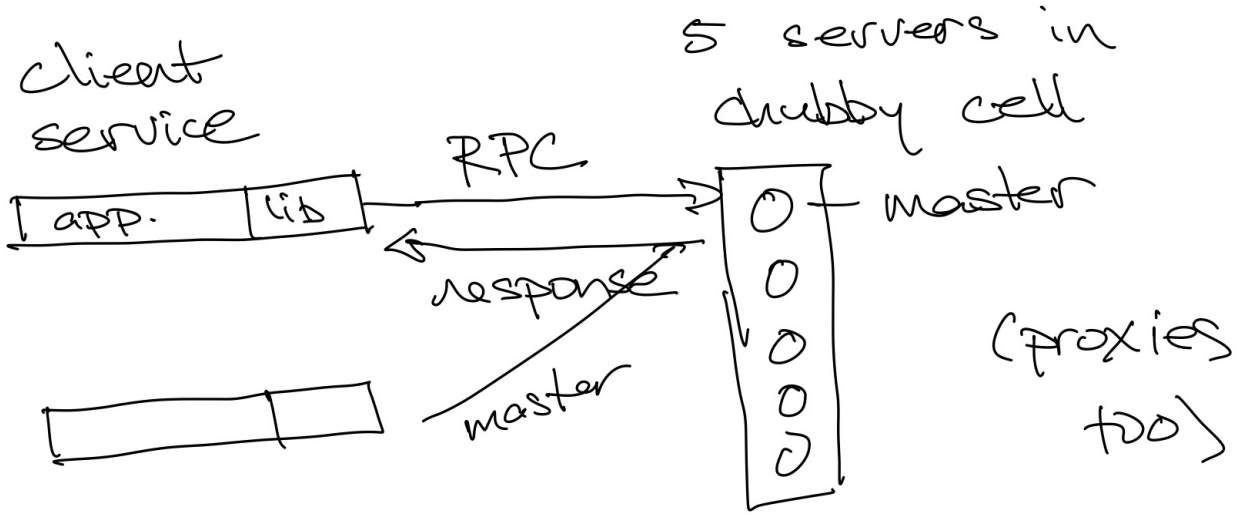


Ok if lock server sometimes not available

- Small delay to acquire a lock

Not ok to lose locks on lock server failures

Lock Service: System Structure



How to contact master? Contact any, redirect to master

Write? Send to all replicas (RSM), wait ack from majority bef. respond to client

Read? Goes to master (different than ZooKeeper!)

Where should replicas be located? Keep local, but different racks (faults)

How to pick master? Consensus protocol, election, “master lease”

Implication of being master? Extra work, bottleneck, limits scalability, same hardware

Details: Master Lease

Why is a master lease needed? Problem if read from master

Possible Fixes?

1. Make reads a consensus operation on RSM (very expensive)
2. Read/ping from majority of nodes (still expensive)
3. Lease: Replicas promise will not elect new leader for lease time
   - Leader periodically renews lease with majority of followers
   - No other leader can be elected during lease
   - Use conservative lease time

Solution in ZooKeeper? Read from followers, stale data is fine

API: File System Interface

Advantages

+ Familiar
+ Services want to store data → same service

  Require whole file read/write → limit to small files only

Hierarchical name space

- /ls/<cell>/dir/file
- Contact local cell by default

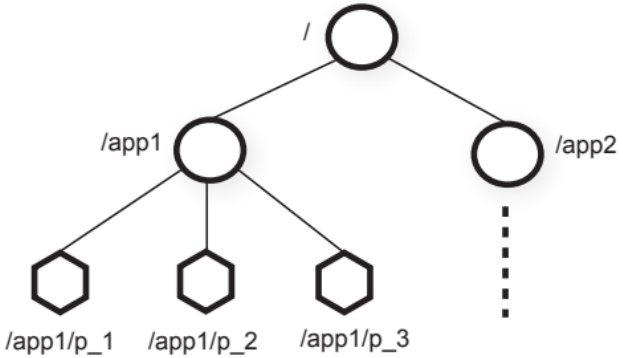
Limitations on API/name space?

- No movement across cells (masters)
- No directory modification timestamps
- No last-access timestamps (helps with caching)
- No links

Nodes: Permanent and Ephemeral

Ephemeral == Temporary

- Node only exists when client has file opened
- **Why is this useful?** Presence indicates client is alive



Per-node meta-data

- Instance number (for this node name)
- Content generation # (increment on each write)
- **Lock generation # (increment on each acquire)**
- ACL generation #
- File content checksum; **handle corruption (not just fail-stop)**

File handle: State associated with **open** files

- Check digits so can't guess
- Sequence # (per master instance)
- Info to recreate on new master

Locks: Per-file

- Two modes: Exclusive lock (write) or shared lock (readers)
- Advisory (not mandatory); **why?**
  - Ignore for debugging or admin
  - Little benefit to mandatory locks – assume cooperative/good developers who check
    - Malicious clients can overwrite data anyways

Complexities of Locks + Failures

Example: Implementing dist fs as a client service (remember coarse-grained locks)

P1: acquire(lock); RPC op(); release(lock);
P2: acquire(lock); RPC op(); release(lock);

What could go wrong?

- P1 fails after acquire and after sending RPC op()
- System correctly aborts P1's lock
- P2 acquires lock
- P1's op() arrives and is executed though P2 now has lock → inconsistency

Solution 1: Use lock sequence #: <lock name, mode, generation #>

P1: seq1 = acquire(lock); RPC op(seq1); release(lock);
P2: seq2 = acquire(lock); RPC op(seq2); release(lock);
Server executing RPC op checks with lock server if seq is still valid (lock still held); if not, reject

Solution 2: Use lock delays (if didn't write service with sequence #)

- Common case: Release(lock) – lock available immediately to be acquired
- But if release(lock) due to failure, impose lock delay before can be re-acquired
- **Length of delay?** Long enough to ensure no late messages (e.g., 1 minute)

Event Notifications: Associated with Nodes

What might clients want to be notified of?

1. File contents modified (location of service changed)
2. Child node added/deleted (file mirroring or membership)
3. Master failover (events may be lost)
4. Handle invalid (failure)
5. Lock acquired by someone (primary elected)
   Not used; why?
6. Conflicting lock request – not used

Event notifications delivered after action completed

- Clients guaranteed to observe new contents

API and Example Usage: Election

- Open(events that should be delivered), close()
- Get-contents() + stat()
- Set-contents(); **What is needed for atomic cmp-and-swap?**
- Lock-acquire(), lock-release()
- Get, set, check-sequencer() of lock

How to implement primary election in some client service?

```
X = open("/ls/cell/dfs/primary", event=notify if contents change, notification_addr);
Notification:
if (try_acquire(x) == success)
    set_contents(x, my-addr);
Else
    primary = get_contents(x);
```

API and Example Usage: Membership

- Open(events that should be delivered), close()
- Get-contents() + stat()
- Set-contents(Content generation for atomic cmp-and-swap)
- Lock-acquire(), lock-release()
- Get, set, check-sequencer() of lock

How can each client i determine if some other client failed?

X = open("/ls/cell/dfs/", event=notify if child deleted , notification_addr);

Open("/ls/cell/dfs/client-i");

Each client i creates ephemeral node with its name in parent service directory

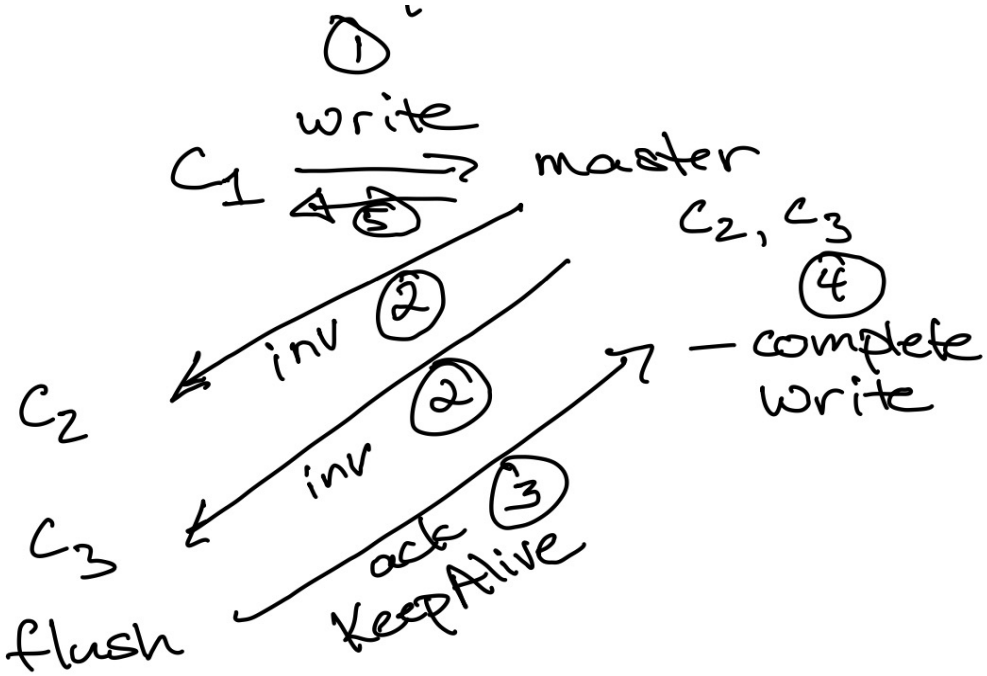
Each client i subscribes to events on parent service

If client j's session ends, client j node deleted, others notified

Caching with Strong Consistency

Motivation: Clients cache data and metadata to reduce server traffic

- Traffic is read-mostly; Write-through
- Lease with **invalidations** (server tracks clients caching); do not **update – why?**



Non-responsive clients?
Need to wait for lease expiration;
Kill session

Read after step 4?
Can return new data

Reads between step 1 and 4?

1) Immediately return old data (tell client can't cache)
2) Block read, don't reply until write completes; return new data

ZooKeeper: Notifications of Changes?

ZooKeeper does not have open()

How does it know which clients to contact?

Clients explicitly ask for “watches”

What is sent when node changes?

Notification of invalidation (not an update)

How long will notifications be sent?

Just 1 notification

Sessions and Keep-Alives

Session: cell ←→ client

- Ensures handles, locks, cached data all valid during session (unless notified)
- Associated lease: Master agrees to not invalidate session unilaterally before lease terminates
  - Master can extend lease or not

Common case: Lease extended

- Use Keep-Alive; always have 1 waiting at master from each client
- Master uses reply for all communication with clients:
  - Extensions
  - Notifications
  - Invalidations
- Advantages:
  - Client must ack invalidations
  - Reply works well with RPC flow of client-server

Implementation of Chubby Service

Replicated database with distributed consensus protocol

Snapshot state periodically to GFS (read later)

Mirroring:

- inform of change
- files are small

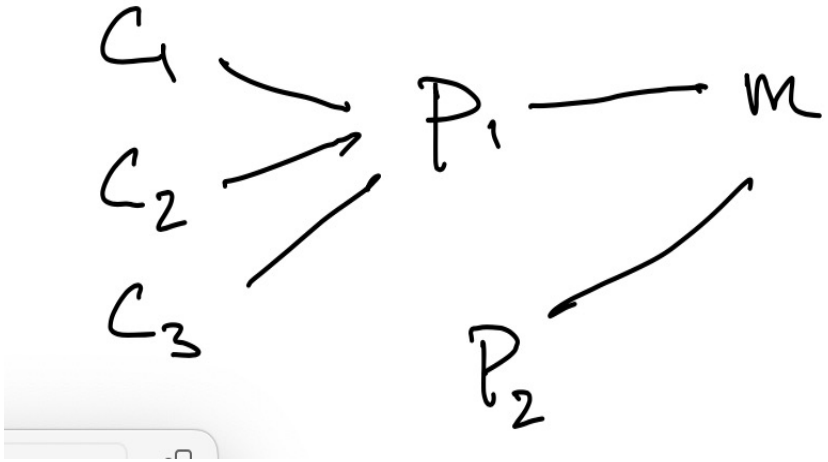
Scaling with Clients

Handles about 90,000 clients (more clients than machines)

Problem? 1 master/cell; master can be overwhelmed

Solutions?

1. Create more cells
2. Increase lease time (fewer keep-Alives, but recovery time increases)
3. Client caching
4. Proxies
   - + Extra caching
   - + Reduce number of keep-alives
   - - Doesn't help writes
   - - Adds latency
   - - More points of failure



Use and Behavior

Interesting observations?

Many active clients
Many cached files
Most files are small
Most RPCs are keep-alives
Negative caching important (otherwise keep asking)
Not that many locks…

time since last fail-over	18 days
fail-over duration	14s
active clients (direct)	22k
additional proxied clients	32k
files open	12k
naming-related	60%
client-is-caching-file entries	230k
distinct files cached	24k
names negatively cached	32k
exclusive locks	1k
shared locks	0
stored directories	8k
ephemeral	0.1%
stored files	22k
0-1k bytes	90%
1k-10k bytes	10%
> 10k bytes	0.2%
naming-related	46%
mirrored ACLs & config info	27%
GFS and Bigtable meta-data	11%
ephemeral	3%
RPC rate	1-2k/s
KeepAlive	93%
GetStat	2%
Open	1%
CreateSession	1%
GetContentsAndStat	0.4%
SetContents	680ppm
Acquire	31ppm

ZooKeeper Similarities and Differences

Similarities?

- Use cases and motivation
- File API including ephemeral nodes
- Sessions
- Notifications
- Read/write whole files (small data)
- Caching for many clients

Differences?

- No locks (can implement with API)
- No open() and close() – Less to track in server, just watches
- Read from any follower
- Pipeline requests from clients (multiple outstanding ops)

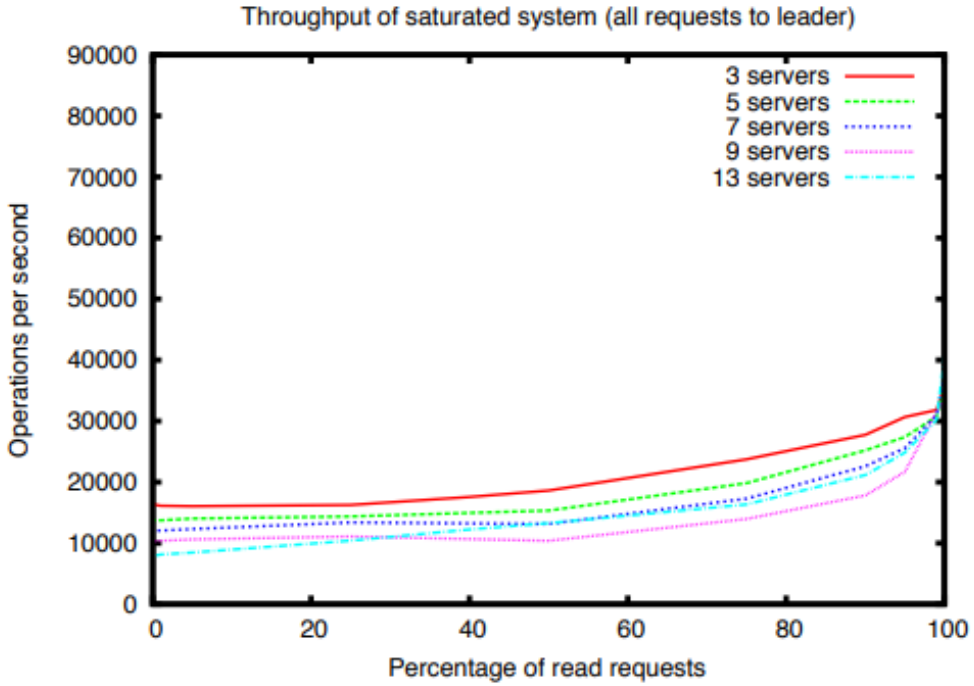
ZooKeeper: Differences in Assumptions

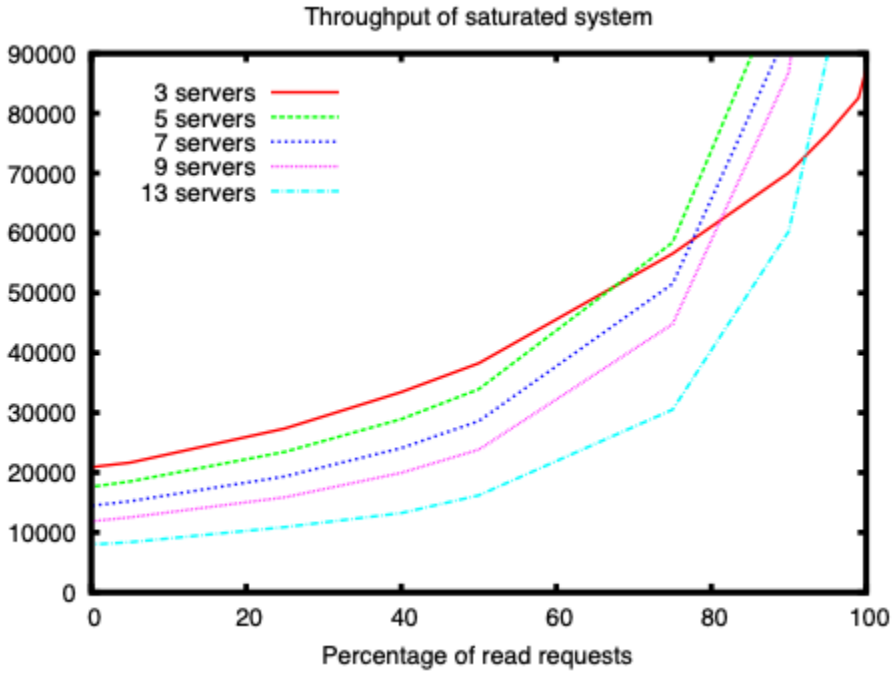
+ Performance
  - Multiple outstanding operations per client
    - Update in pipeline

- Consistency

Impact on Reads?
  - Can be sent to any node of ZooKeeper
  - Writes: Linearizable
  - Reads: FIFO order per client (use explicit sync if need linearizability)

Performance: Reading Leaders or Followers





Observations?

Throughput increases w/ % reads (less work) in both systems

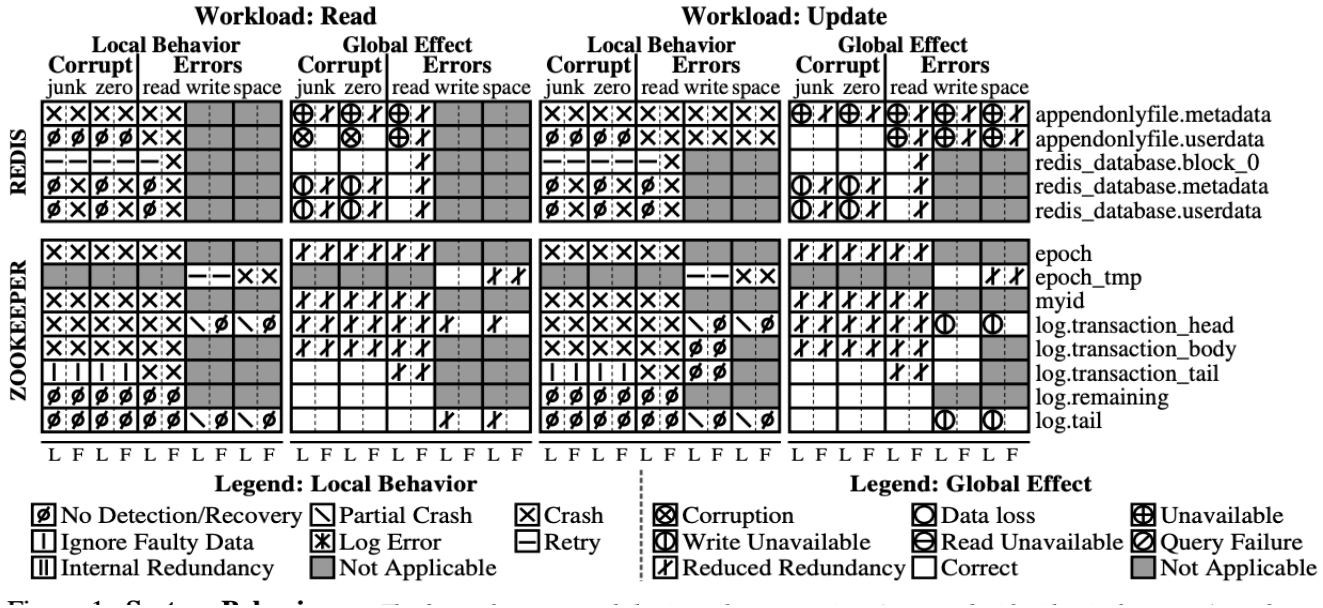
All writes, similar throughput for both;
throughput worse w/ more servers

ZooKeeper: More reads %: Throughput increases dramatically

With 100% reads: Better throughput w/ more servers

Servers	100% Reads	0% Reads
13	460k	8k
9	296k	12k
7	257k	14k
5	165k	18k
3	87k	21k

ZooKeeper: Robust to Corruption?



Observations? Global effects? Local behavior?

Even with data corruptions (not fail-stop), decent fault tolerance

Global effects: Correct, Reduced Redundancy, just few Write Unavailable

Why?

Local behavior: Full Crashes are fine – new leader elected reliably, eventually repair log by contacting new

Cases where detection kills only one thread, KeepAlive thread remains, leads to WriteUnavailable

Conclusion: Coordination Services in Practice

Consensus → Service abstraction

- Chubby and ZooKeeper make Paxos/Raft *usable* for systems builders

Design point matters

- Chubby: simpler model, strong consistency, centralized (leader-focused)
- ZooKeeper: more flexible API, higher performance options, more developer responsibility

Consistency vs. performance is not binary

- Both systems *selectively relax or enforce* guarantees (e.g., reads, caching, leases)
- Choose where to use expensive, precise leader-based ops

APIs shape system behavior

- Naming, ephemeral nodes → guide how developers build distributed systems

Real systems need more than "just consensus"

- Failure handling, caching, session semantics, edge cases dominate the design

Widely-reused building blocks

- *Infrastructure* that many other services depend on